

人工智能与机器学习

张建明 徐巍华

智能系统与控制研究所

杭州·浙江大学·2024

人工智能

需掌握的核心内容

• 人工智能概述

- 人工智能概念
- 智能体概念
- 人工智能的历史
- 人工智能的研究

• 理性智能体

- 智能体与任务环境
- 理性的概念
- 智能体的结构与类型

• 问题求解

- 基于目标的智能体
- 问题的形式化
- 基本的搜索方法
- 盲目搜索
- 信息搜索
- 对抗搜索
- 启发式函数
- α - β 剪枝

• 知识表示与推理

- 基于知识的智能体
- 知识的概念
- 知识表示方式
- 逻辑
- 蕴涵
- 有效性与可满足性
- 命题逻辑表示与推理
- 一阶逻辑表示与推理
- 量词的实例化
- 合一的概念
- 归结的概念与应用

绪论

- 什么是人工智能
- 人工智能的基础
- 人工智能发展史
- 人工智能的研究与应用



人工智能：Artificial Intelligence

(简称为 AI)

人工智能概念的一般描述

“人工智能”一词目前是指用计算机模拟或实现的智能，研究如何在机器上实现人类智能。即用机器来模仿人的智能。因此人工智能又称机器智能。

能力方面：人工智能是智能机器所执行的通常与人类智能有关的功能，如判断、推理、证明、识别、感知、理解、设计、思考、规划、学习和问题求解等思维活动。

学科方面：是一门研究如何构造智能机器或智能系统，以模拟延伸和扩展人类智能的学科。

智能 = 知识 + 思维

人工智能是研究人类智能活动的规律，构造具有一定智能的**人工系统**，研究如何让**计算机**去完成以往需要人的智力才能胜任的工作，也就是研究如何应用计算机的软硬件来模拟人类某些智能行为的基本理论、方法和技术。

- **智能系统**
- **智能机器**

智能包含的能力（四类能力）

感知能力

记忆和思维能力

- 抽象思维
- 形象思维
- 灵感思维

学习和自适应能力

行为能力

含义：是人们对感知到的外界信息作出动作反应的能力。

信息来源：由感知直接获得的外界信息经过思维加工后的信息。

实现过程：通过大脑（脊髓）来控制由语言表情、体姿等来实现。

组织成四类的人工智能的若干定义

Stuart Russell和Peter Norvig把已有的一些人工智能定义总结为4类

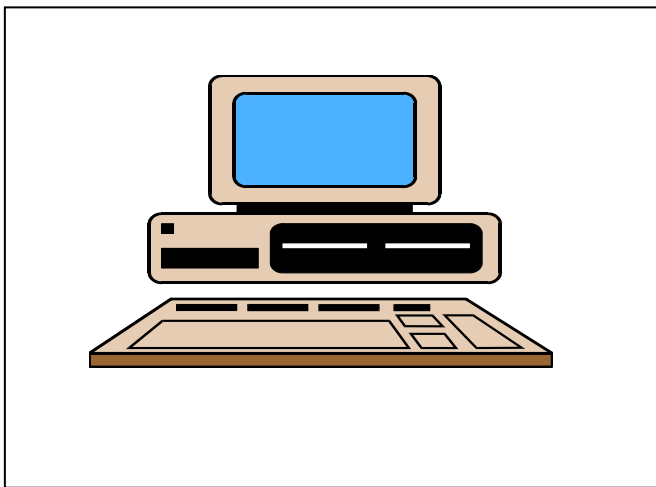
(2003年，人工智能：一种现代方法)：

	根据与人类表现的逼真度来衡量是否成功	依靠“合理性”指标来衡量是否成功
关注思维过程与推理	像人一样思考	合理地思考
	"使计算机思考的令人激动的新成就,.....按完整的字面意思就是:有头脑的机器"(Haugeland,1985) "与人类思维相关的活动,诸如决策、问题求解、学习等活动[的自动化]"(Bellman,1978)	"通过使用计算模型来研究智力"(Charniak和McDermott,1985) "使感知、推理和行动成为可能的计算的研究"(Winston,1992)
强调行为	像人一样行动	合理地行动
	"创造能执行一些功能的机器的技艺,当由人来执行这些功能时需要智能"(Kurzweil,1990) "研究如何使计算机能做那些目前人比计算机更擅长的事情"(Rh和Knight,1991)	"计算智能研究智能 Agent的设计。"(Pole等人,1998) "AI...关心人工制品中的智能行为。"(Nilsson,1998)

类人行为（像人一样行动）：图灵测试方法

Turing (1950) “computing machinery and intelligence”

- ⑩ “机器可以思考吗？” → “机器可以产生智能行为吗？”
- ⑩ 测试智能行为的操作型试验
- ⑩ 包含AI的主要内容：知识表示，自动推理，自然语言理解，机器学习，机器视觉，机器人技术



智者



询问者

理性智能体

智能体 (agent) 是指能够感知和动作的实体

简单说，一个智能体就是从感知序列到动作的一个函数：

$$f : P^* \rightarrow A$$

对于任何给定的环境和任务，来设计具有**最佳性能**的智能体（或主体集）

注意：**计算能力有限使得最完美的智能体不可实现**→只能针对给定的机器资源
设计最好的**程序**

AI的诞生 (1956年)

AI诞生于一次历史性的聚会

- **时间:** 1956年夏季 (2个月)
- **地点:** 达特莫斯 (Dartmouth) 大学
- **目的:** 为使计算机变得更“聪明”，或者说使计算机具有智能
- **会议结果:** 由麦卡锡提议正式采用了“Artificial Intelligence”这一术语
- 此后，美国形成了多个人工智能研究组织，如纽厄尔和西蒙的Carnegie RAND协作组，明斯基和麦卡锡的MIT研究组，塞缪尔的IBM工程研究组等。

1956 Dartmouth Conference: The Founding Fathers of AI



John McCarthy



Marvin Minsky



Claude Shannon



Ray Solomonoff



Alan Newell



Herbert Simon



Arthur Samuel



Oliver Selfridge



Nathaniel Rochester



Trenchard More

Founding fathers of AI. Courtesy of scienceabc.com

从典型任务/应用对人工智能分类 (上世纪70年代以来)

典型任务/应用	模仿对象
机器定理证明（逻辑和推理）	解题者
机器翻译（自然语言理解）	译者
专家系统（问题求解和知识表达）	专家如医生
博弈（树搜索）	弈者
模式识别（多媒体认知）	认知者
学习（神经网络），	初学者
机器人和智能控制（感知和控制）	生物者

■ 形成了符号学派、连接学派、行为学派

人工智能学派及其认知观

符号主义(Symbolicism)，又称为**逻辑主义(Logicism)**、**心理学派(Psychlogism)**或**计算机学派(Computerism)**

认为人的认知基元是符号，而且认知过程即符号操作过程。认为人是一个物理符号系统，计算机也是一个物理符号系统，因此，我们就能够用计算机来模拟人的智能行为。知识是信息的一种形式，是构成智能的基础。人工智能的核心问题是知识表示、知识推理和知识运用。

联结主义(Connectionism)，又称为**仿生学派(Bionicsism)**或**生理学派(Physiologism)**

认为人的思维基元是神经元，而不是符号处理过程。认为人脑不同于电脑，并提出联结主义的大脑工作模式，用于取代符号操作的电脑工作模式。

行为主义(Actionism)，又称**进化主义(Evolutionism)**或**控制论学派(Cyberneticsism)**

认为智能取决于感知和行动。认为智能不需要知识、不需要表示、不需要推理；人工智能可以象人类智能一样逐步进化。智能行为只能在现实世界中与周围环境交互作用而表现出来。符号主义、联结主义对真实世界客观事物的描述及其智能行为工作模式是过于简化的抽象，因而是不能真实地反映客观存在的。

人工智能的三个阶段

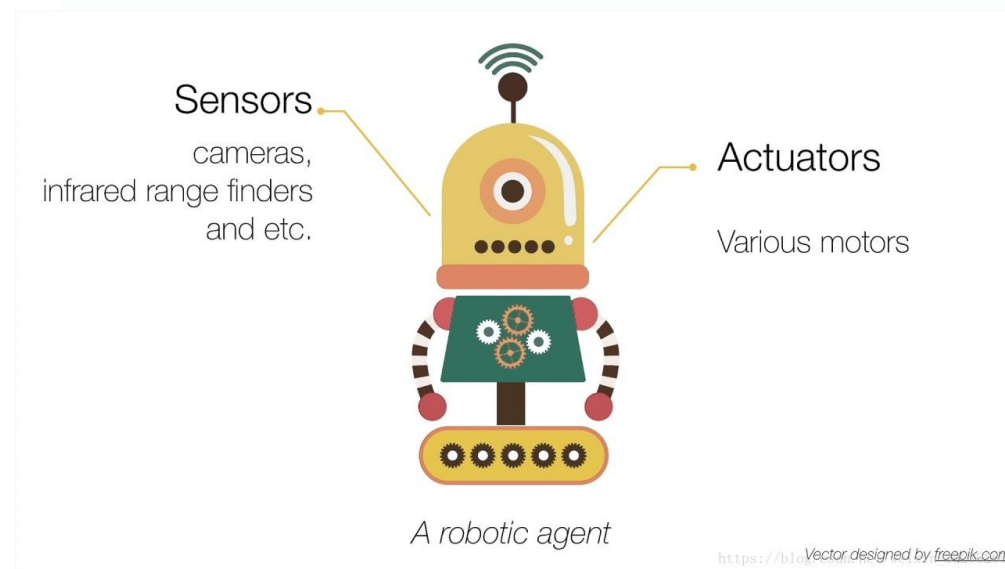
- 1、能存会算的计算智能：仅仅局限于计算机的存储与运算的满足
- 2、能听会说、能看会认的感知智能：通过传感器和算法感知世界
- 3、能理解会思考的认知智能：能够对自然和人类世界进行认知



机器学习是一种实现人工智能的方法
深度学习是一种实现机器学习的技术

第二章 智能AGENT（智能体）

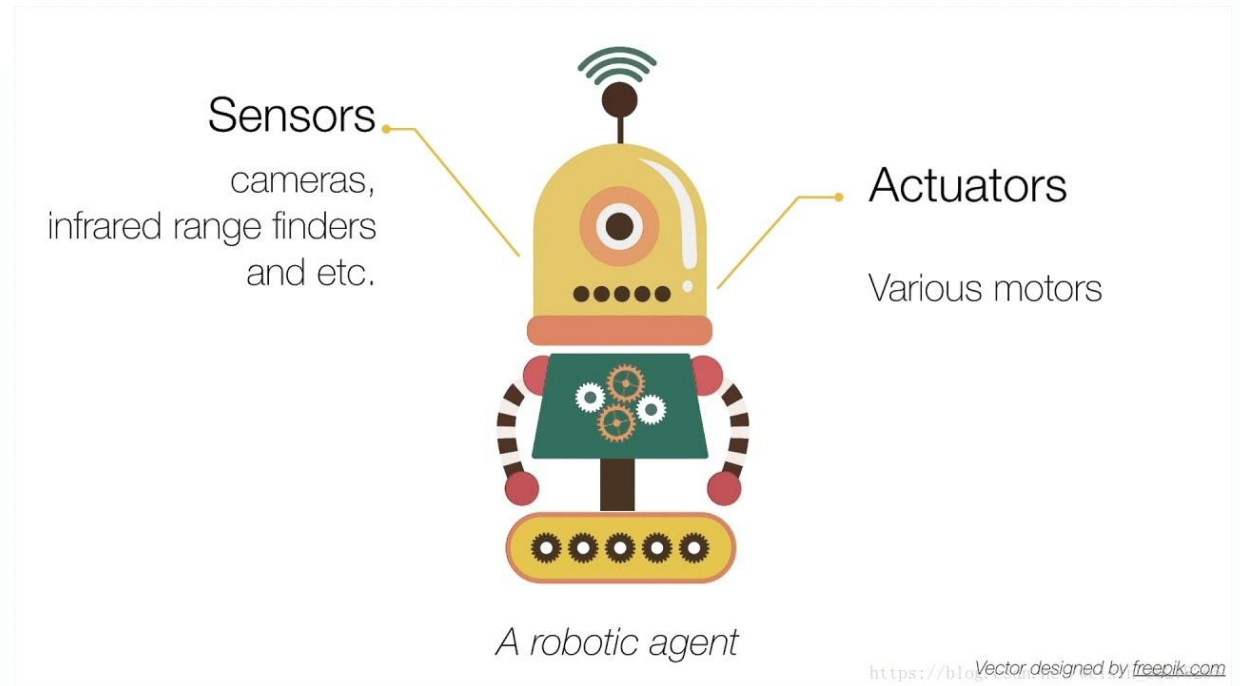
- Agent基本定义（Agent的本质）
- 评价Agent行为（Agent是否完美）
- 任务环境（环境的多样性）
- Agent的结构（Agent分类）



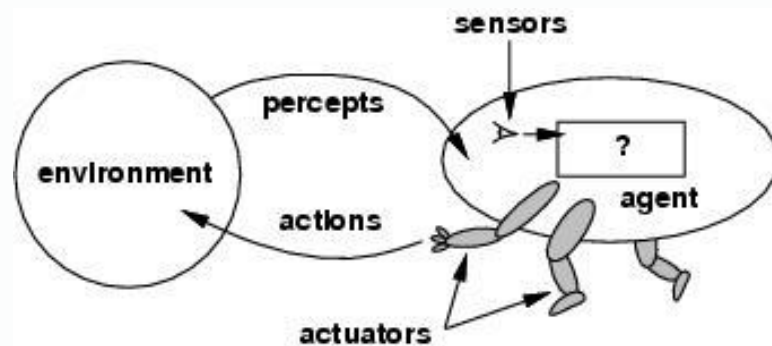
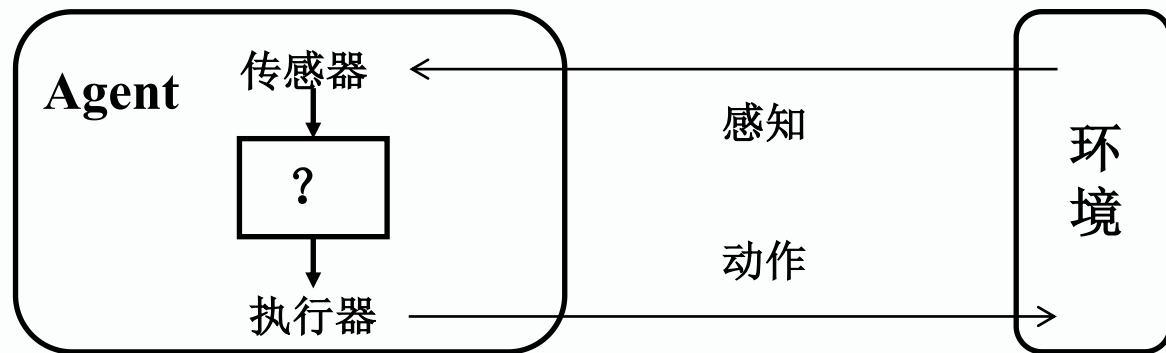
智能体 (Agent) 基本定义

- 智能体 (Agent) : 能够**感知**和**动作**的实体
(任何独立的能够思想并可以同环境交互的实体都可以抽象为智能体)

- 人类智能体
- 机器人智能体
- 软件智能体
-



智能体和环境



Agent 通过传感器感知环境，并通过执行器对所处环境产生影响。问号（？）表示从感知到动作的映射函数

简单说，一个智能体就是从**感知序列**到**动作**的一个**函数**： $f : P^* \rightarrow A$

感知信息：表示任何时刻Agent的感知输入。

感知序列：Agent收到的所有输入数据的完整历史。

Agent在任何时刻的行动选择，取决于到该时刻为止的整个感知序列。

Agent函数：将任意给定感知序列映射到Agent的动作。
可以描述Agent的行为。

Agent函数：从外部反映了Agent的特性

Agent程序：从Agent内部来看，Agent函数是通过Agent程序实现的

Agent函数：是一个抽象的数学表示

Agent程序：是Agent函数在某种物理体系上的具体实现

理性智能体 (Rational Agent)

感知序列	行动
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
...	...

- **理性Agent是做事正确的智能体。**
 - 即Agent函数表格的右边都填写正确。
- **什么是“正确的事”？**
 - 需要某种方法度量Agent成功与否，连同对环境、传感器、执行器的描述，为Agent 面临的任務提供一个完整的规范说明。

- 修改表格的右边就可以定义不同的真空吸尘器世界Agent。

问题：

- **怎样填表才是最好的算法？**
- **如何判断Agent是 好的、坏的、智能的、愚蠢的？**

性能度量：Agent成功程度的标准

性能度量：通常由理性Agent的**设计者**给出，根据实际在所处的环境中**希望得到的结果**来设计度量，而不是根据智能体**应该表现的行为**。

当把Agent置于一个环境中后，它将针对收到的感知信息产生动作序列。

该动作序列引起环境历经一个状态序列。

如果该状态序列是想要的，则Agent的性能良好。

- 需要客观的性能度量（通常由理性Agent的设计者给出）
- 没固定的适合所有智能体的度量标准
- 理性Agent应该**选择期望能使其性能度量最大化的行动**

理性 (Rationality) 的判断



理性Agent

对于每一个可能的感知序列，根据已知的感知序列和内建的先验知识，理性Agent选择能使性能指标的期望值最大化的动作。

任务环境的性质

- **完全可观察与部分可观察**

- 完全可观察：智能体能获取环境的完整状态，智能体不需要内部维护来记录世界的状况

- **确定性与随机**

- 确定性：环境的下一个状态完全取决于当前状态和智能体的行动
 - 如果环境是确定的，除非有其它智能体活动的影响，那么我们称环境是策略的。

- **片段式与延续式**：行动的选择是否取决于当前片段

- **静态与动态**

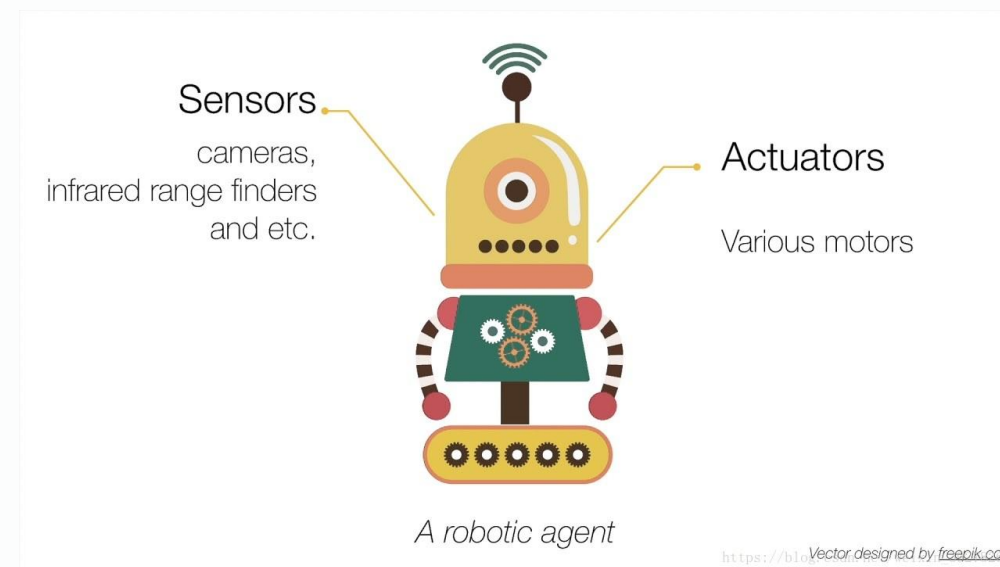
- 环境在智能体思考的时候是否会变化
 - 出租车驾驶是动态的；纵横字谜游戏是静态的
 - **半动态**：环境本身不随时间的流逝而变化，但智能体的性能评价随时间变化；例如计时棋赛。

- **离散与连续**：环境的状态，时间的处理方式，智能体的感知信息和行动

- **单智能体与多智能体**：哪些智能体必须被视为智能体？



- AI的任务是设计智能体**程序**，实现把感知信息映射到行动的智能体**函数** f
- **体系结构**为**程序**提供：
 - 来自**传感器**的感知信息
 - 运行程序
 - 把程序产生的行动送到**执行器**
- 所选择的**程序**必须适合**体系结构**



四种基本的智能体(结构) 类型



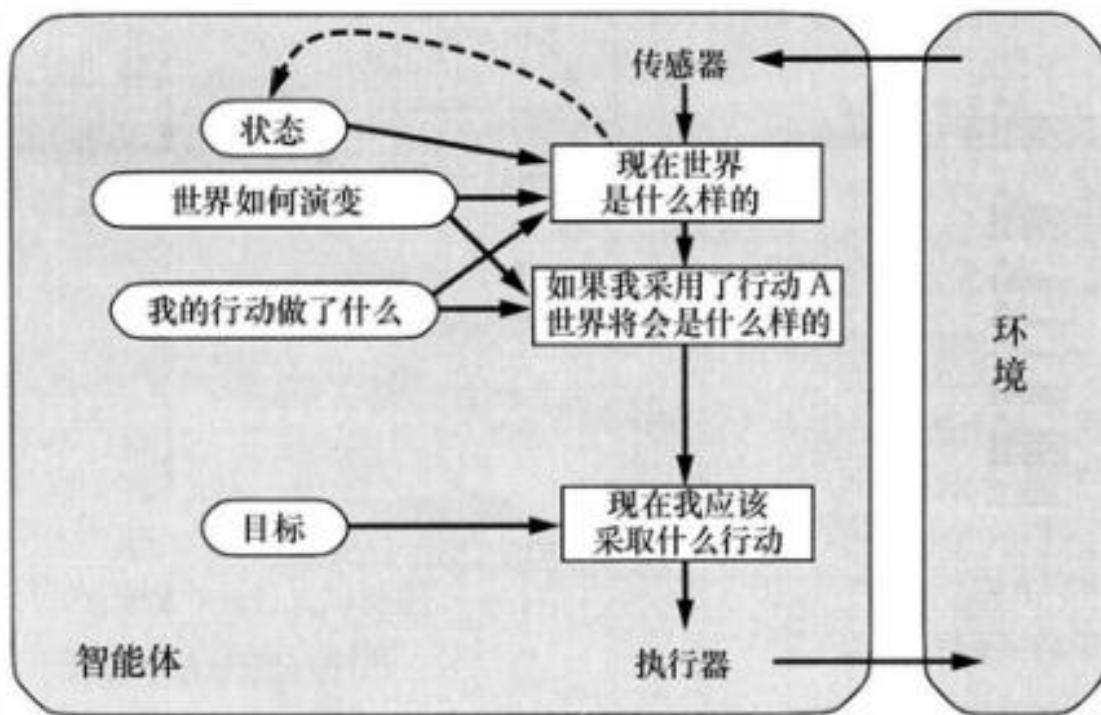
- 每种Agent程序都以**特定的方式**结合了**特定的成分**来生成**行动**
- 所有这些智能体都可以转变为**学习智能体**，能够提高性能以便生成更好的行动
- Agent程序的各组件是如何工作的

第三章 通过搜索进行问题求解

Solving Problems By Searching

- **问题求解**智能体 — 一类基于目标的智能体
- 问题举例
- 通过**搜索**求解
- 基本的搜索算法（无信息搜索策略）

基于目标的智能体Goal-Based Agent

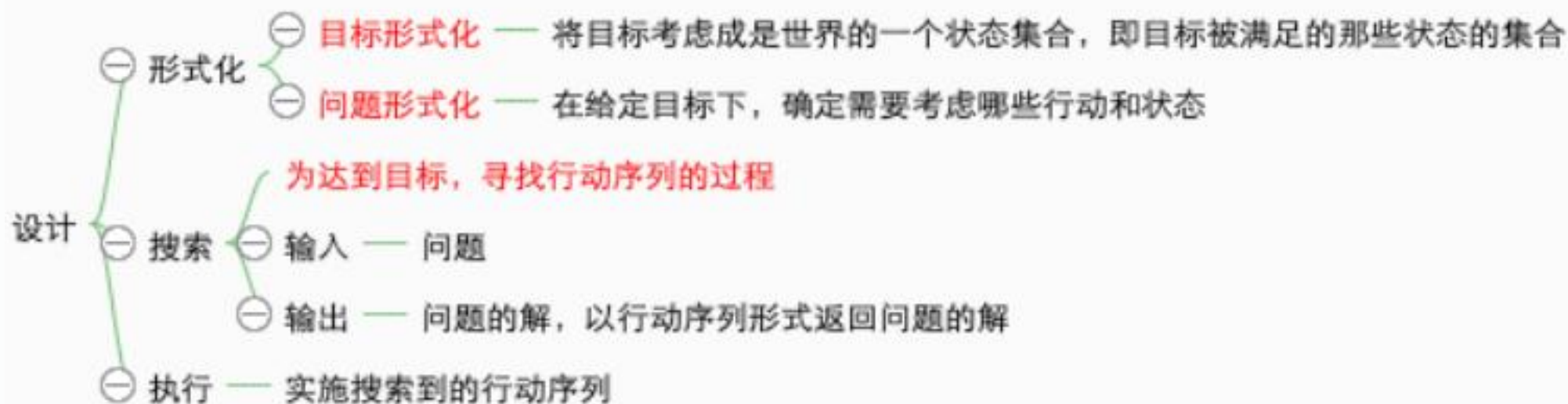


- 智能体不仅需要当前**状态**的描述，还需要**目标**信息来描述要达到的状况
- 智能体把目标信息和可能**行动**的**结果**信息结合起来，以选择达到目标的行动（**决策**）
- **搜索**和**规划**能帮助智能体找到达到目标的行动序列
- 既追踪记录世界的状态，又记录要达到的目标集，并选择能达到目标的行动
- 效率降低，灵活度增加，功能增强

问题求解的步骤

(问题求解智能体设计的基本步骤)

- **目标形式化 (goal formulation)**：“成功”的状态描述
- **问题形式化 (problem formulation)**：根据所给的目标考虑行动和状态的描述
- **搜索 (search)**：通过对行动序列代价计算来选取最佳的行动序列
- **执行 (execute)**：给出“解”执行行动



状态图搜索

- **搜索**：从初始节点出发，沿着与之相连的边试探地前进，寻找目标节点的过程。
- 搜索过程中经过的节点和边，按原图的连接关系，便会构成一个树型的有向图，这种树型有向图称为**搜索树**。
- 搜索进行中，搜索树会不断增长，直到当搜索树中出现目标节点，搜索便停止。这时从搜索树中就可很容易地找出从初始节点到目标节点的路径（**解**）来。

搜索技术： 显式搜索树

搜索树：根节点为初始状态、连线表示行动、结点对应问题的状态

- 基本做法：先追随一个选择，搁置其它的，等到万一发现第一个选择不能通向问题的解时再考虑。

第一步，检测该节点是不是否为目标状态，如In(Arad),选择各种行动。生成一个状态集，{In(Sibiu),In(T),In(Z)}。
接着从其中选择一个考虑，等发现不能求解在选择其他的。

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

一般的生成搜索树算法的描述

问题求解的性能度量

- **完备性：**有解时能否保证找到解
- **最优性：**这个搜索策略是否能找到最优解
- **时间复杂度：**根据搜索过程中产生的节点数目来度量，即找到解所花的时间
- **空间复杂度：**在执行搜索的过程中需要的内存多少
- 时间与空间的复杂度往往要与问题难度的某种度量一起考虑

无信息搜索策略/盲目搜索策略

UNINFORMED SEARCH STRATEGIES

除了问题定义中提供的状态信息外，**没有任何附加信息**（只能利用问题定义中给出的信息）

搜索任务：生成后继，并区分目标状态与非目标状态

根据节点扩展的次序分类

① 广度（宽度）优先搜索

Breadth-first search, BFS

② 一致代价搜索

Uniform-cost Search, DFS

③ 深度优先搜索

Depth-first Search, DFS

④ 有限深度优先搜索（深度受限搜索）

Depth-limited Search, DLS

⑤ 迭代加深搜索(迭代加深的深度优先搜索)

Iterative deepening search, IDS
(Iterative deepening depth-first search, IDDFS)



第三章 问题求解

有信息（启发式）的搜索策略

Informed Search

- 使用问题本身的定义之外的特定知识
- 比盲目搜索更有效

■ 最好优先搜索

■ 贪婪最好优先搜索

■ A*搜索

■ 启发式函数

最好优先搜索 Best-First-Search

(最佳优先搜索) BFS

- **基础：**与一致代价搜索相同，基于广度优先搜索算法
- **核心思想：**与一致代价搜索不同
 - 最佳优先搜索：使用一个 **评价函数 $f(n)$** 给每个节点估计他们的**希望值**。对 f 的选择决定了搜索策略
每次搜索时优先扩展最有希望的未扩展节点（评估值最小的节点） Best-First
 - 一致代价搜索：使用一个 **路径耗散函数 $g(n)$** 扩展路径消耗最低的节点
总是搜索总成本最小的节点： Cheapest-First

启发函数（Heuristic Function）

$h(n)$ = 从 **节点 n 代表的状态** 到 **目标状态** 的 路径耗散 的 最小估计值

例如：

在罗马尼亚问题中最低耗散路径的耗散估计值可以是相应的两个城市之间的直线距离。

A*搜索：最小化总的估计解耗散

思想：避免扩展代价昂贵的节点

评价函数： $f(n) = g(n) + h(n)$

$g(n)$ = 初始节点到达节点n的路径耗散【到达节点n已经发生的实际代价】

$h(n)$ = 从节点n 到目标节点的最小路径耗散估计【从节点n到目标的代价估计值】

$f(n)$ = 从初始节点经过节点n到达目标节点的路径最小耗散**估计**

如果h从不**高估**到达目标的最低路径耗散值，我们称h是**可容纳启发式**。

(admissible, 可容许的, 可容纳的)

An admissible heuristic is one that never overestimates the cost to reach the goal.

迭代深入A* (IDA*)

- IDA*算法就是基于迭代加深的A*算法
- 截断值是超过上一次迭代截断值的节点中最小的f耗散值
- 与A*比起来, IDA*更实用: 1
 - 不需要判重, 不需要排序;
 - 空间需求减少。
- 可以避免节点队列排序的实际系统开销
- 典型应用: 八数码问题
- 困难: 习题3.17

A*改进：递归最好优先搜索(RBFS)

尝试模拟标准的最佳优先搜索而只需线性空间。

1. 记录当前节点的祖先可得到的最佳可替换路径的f值。
2. 如果当前的f值超过了这个限制，则递归将转回到替换路径。
3. 向上回溯改变f值到它的孩子的最佳f 值
4. 重复扩展这个上个节点，因为仍有可能存在较优解。

启发式函数的本质问题探讨

HEURISTIC FUNCTIONS

超越经典搜索

Beyond Classical Search

环境：可观察的，确定的，已知的
问题的解：行动序列
关注点：路径代价

- 无信息搜索
- 有信息搜索
 - 最好优先搜索
 - 贪婪最好优先搜索
 - A*搜索

■ 启发式函数

局部搜索：
考虑对一个或多个状态进行评价和修改，而不是系统地探索从初始状态开始的路径。

■ 局部搜索算法

- 爬山法
- 模拟退火算法
- 遗传算法

关注解状态而不是路径代价

更接近于现实世界

对抗搜索

- 完全信息博弈
 - 极小极大决策
 - α - β 剪枝
 - 资源有限
- 概率博弈
- 不完全信息博弈

本章讨论在有其他Agent计划与我们“对抗”时，该如何预先规划的问题。

人工智能中的博弈

- **人工智能中的博弈**：有完整信息的、确定的、轮流行动的两个游戏者的零和游戏。（可为多人）
 - 博弈是**多智能体**环境，包含竞争与合作
 - 博弈很有趣，与人类智慧密不可分
 - 博弈很难，求解通常是NP-Hard，注重时间效率
 - 博弈的状态表达容易，动作数量少，规则严谨
 - 博弈的抽象本性成为AI研究者感兴趣的对象

- **博弈：双方**的智能活动，任何一方都不能单独控制博弈过程，而是由双方**轮流**实施其控制对策的过程。

- 参与者轮流出招，可以将其写成类似于路径寻找的问题：

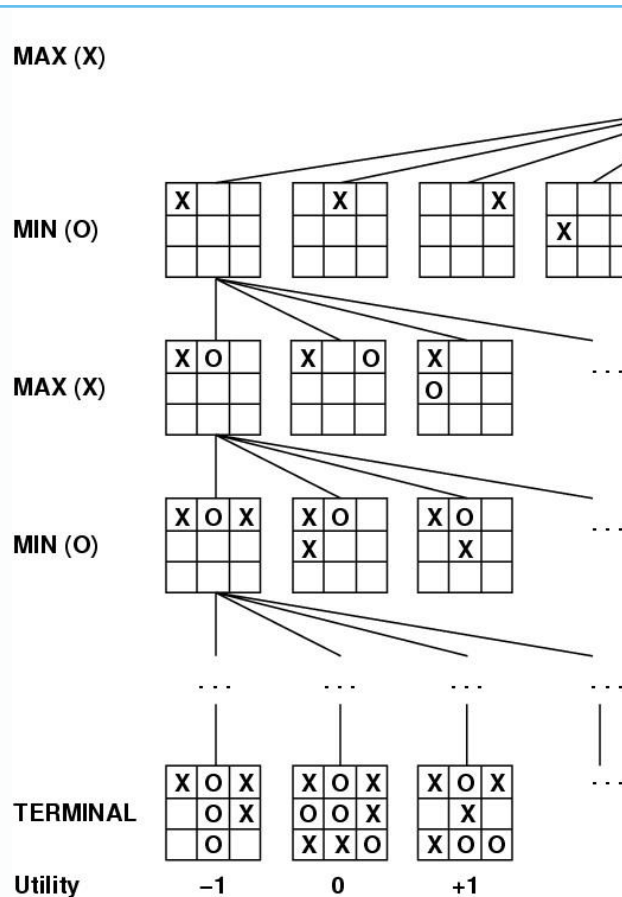
- **初始状态**：包括棋盘局面和确定该哪个游戏者出招
- **后继函数**：返回(move, state)列表，每一项表示一个合法招数和对应的结果状态。
- **终止测试**：判断游戏是否结束。游戏结束的状态称为终止状态。
- **效用函数**：也称目标函数或收益函数，是终止状态的得分。国际象棋中赢、输、平分别是1， -1和0分，而围棋、黑白棋等可以有更多的结果。

- S_0 初始状态
- $Player(s)$ 谁行动
- $Action(s)$ 状态s下的合法移动集合
- $Result(s,a)$ 转移模型
- $Terminal-test(s)$ 终止测试，游戏是否结束
- $Utility(s,p)$ 效用函数

博弈中的优化策略

Optimal Decisions in Games

- **一般的搜索问题：**最优解是达到目标状态的行动序列（终止状态就是取胜）。
- **在对抗搜索中：**在**每一步**的时候，MAX/MIN如何来**设计评估函数**来选择**“最利于”**自己的下棋步骤？比如在第一步时，MAX到底该如何得知自己要选择9个选择中的哪一个。



这个图列举了双方能接受的**所有**选择。我们可以看到**只有叶子节点才有评估函数**，可以看到从根节点到叶子节点是双方按照当前路径走下来的最终结果（赢、输、平局）。**每条路径都对应一个结果，双方不论在什么时候，肯定都要选择“最利于”自己获胜的步骤。**

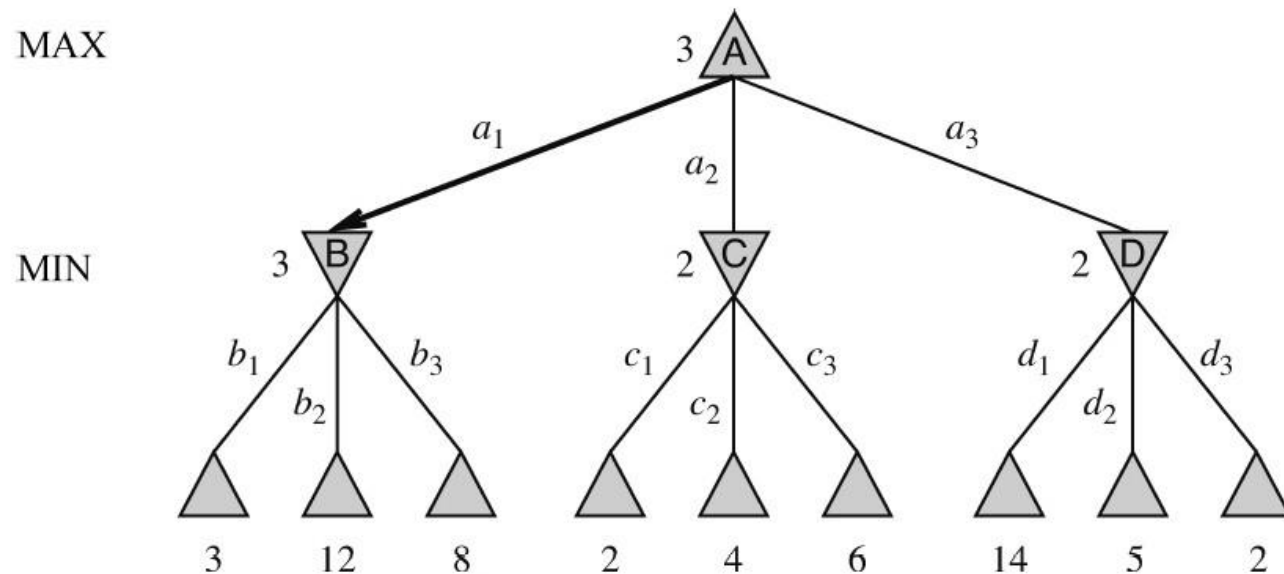
评估函数、效用函数：“赢面”

最优策略的确定

- 对MAX方来说，评估函数越大越好，而对MIN方来说，评估函数越小越好。也就是在每一步中，MAX方选择所有节点中评估函数最大的节点，作为自己当前的落棋选择，而MIN则相反。
- 通过检查每个节点的**极小极大值**来决定，记为MINMAX-VALUE(n)。

$$MINIMAX - VALUE(n) =$$

$$\begin{cases} UTILITY(n), & \text{if } n \text{ is a terminal node} \\ \max_{s \in Successors(n)} MINIMAX - VALUE(s), & \text{if } n \text{ is a MAX node} \\ \min_{s \in Successors(n)} MINIMAX - VALUE(s), & \text{if } n \text{ is a MIN node} \end{cases}$$



两层博弈树

Δ 结点是“MAX 结点”，代表轮到 MAX 走， ∇ 结点是“MIN 结点”。终止结点显示 MAX 的效用值；其他结点标的是它们的极小极大值。MAX 在根结点的最佳行棋是 a_1 ，因为它指向有最高的极小极大值的后继，而 MIN 此时的最佳行棋是 b_1 ，因为它指向有最低的极小极大值的后继

https://blog.csdn.net/weixin_45552760

给定一棵博弈树，最优策略可以通过检查每个节点的极小极大值来决定

- MAX将优先选择移动到一个具有**极大值**的状态
- MIN则选择移动到具有**极小值**的状态

都是相对MAX而言的

α - β 剪枝

AlphaBeta剪枝方法是对Minimax方法的优化
它们产生的结果是完全相同的，只不过运行效率不一样

基本思想

根据上一层已经得到的当前最优结果，决定目前的搜索是否要继续下去

当你知道你有一个选择A时，此时你知道了B选择不如A选择好，那么你就不需要知道B选择有多坏。

2个假设：

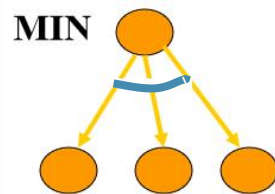
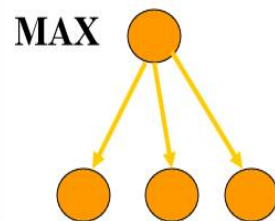
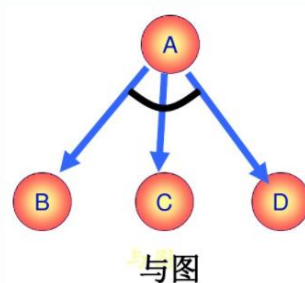
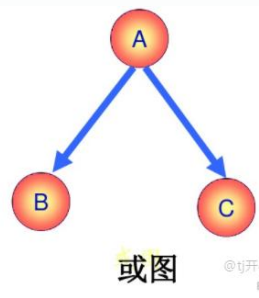
- 整个博弈过程属于**零和博弈**，即一方的收益必然意味着另一方的损失，博弈双方的收益和损失相加总和永远是零，双方不存在任何合作的可能。粗俗理解就是你赚了我就亏了。
- 博弈双方**足够聪明**，即每一方在决策时总会选择使自己利益最大化的决策。粗俗理解就是，假设是在下棋，那么大家下棋时大家走的都是固定的且是對自己最好的，下多少局也是这样。

基本概念

从MAX方的角度来看：

- 所有MAX方的节点都是“或节点”（因为扩展MAX方节点时，MAX方可选择扩展最有利于自己的节点，只要可扩展的子节点中有一个对己有利，则该节点就对己有利。）

- 所有MIN方节点都是“与节点”（因为MIN方必定选择最不利于MAX方的方式来扩展节点，只要MIN方节点的子节点中有一个对MAX方不利，则该节点就对MAX方不利，故为“与节点”。）



• 定义：

- α 值：有**或后继**的节点（**或节点**），取当前子节点中的最大倒推值为其下界，称为 α 值。节点倒推值 $\geq \alpha$ ；
- β 值：有**与后继**的节点（**与节点**），取当前子节点中的最小倒推值为其上界，称为 β 值。节点倒推值 $\leq \beta$ ；

与节点、或节点 隔层交替出现

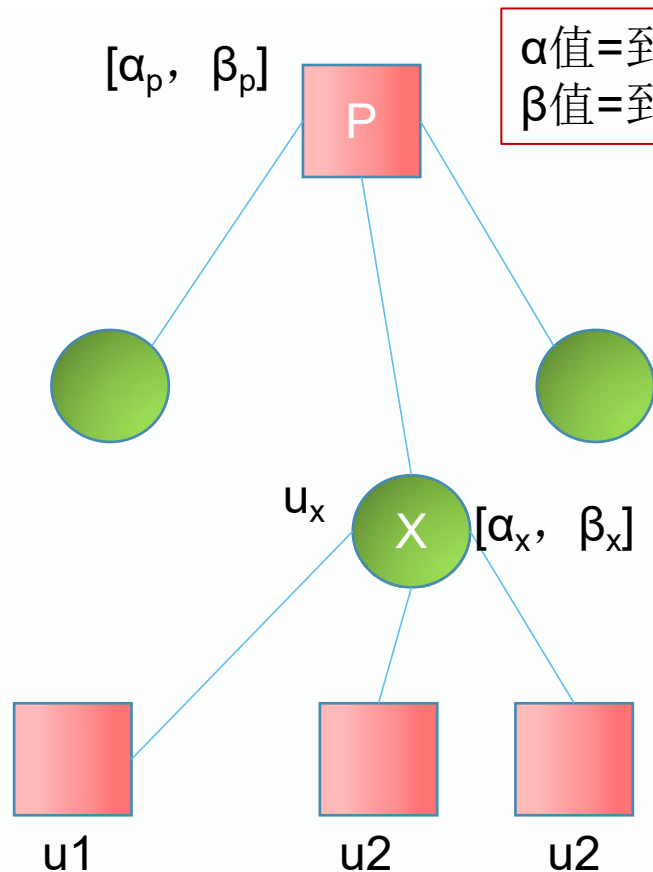


MAX



MIN

或中取大，与中取小



α 值=到目前为止路径上发现的MAX的最佳（即极大值）选择
 β 值=到目前为止路径上发现的MIN的最佳（即极小值）选择

- **β 剪枝**: 节点X的 α 值不能降低其父节点的 β 值 ($\alpha_x > \beta_p$) , X以下的分支可停止搜索, 且X的倒推值为 α
- **α 剪枝**: 节点x的 β 值不能升高其父节点的 α 值 ($\beta_x < \alpha_p$) , X以下的分支可停止搜索, 且X的倒推值为 β
- **α - β 剪枝**: 保存 α 、 β 值, 并且当可能时进行剪枝的整个过程

对MIN节点X, 其效用值
 $u_x = \min(u_1, u_2, u_3)$, $\alpha_x = u_x \leq \beta_x$
 对MAX节点P, 其效用值
 $u_p = \max\{u_x\}$, $\alpha_p \leq u_p = \beta_p$

- 剪枝不会影响最终的结果
- α - β 剪枝的效率很大程度上取决于**检查后继的顺序**；
好的移动排放次序能够提高剪枝的效率
- “理想排序”下：先检查那些可能最好的后继，时间复杂度从 $O(b^m)$ 减小到 $O(b^{m/2})$
- 最佳节点排序：不可实现
 - 静态评估函数法
 - 迭代加深

如何设计评估函数？

- 应该以和真正的效用函数同样的方式对终止状态进行排序。
- 评估函数的计算不能花费太多的时间。
- 对于非终止状态，评估函数应该和取胜的实际机会密切相关。
- 在计算能力有限情况下，评估函数能做到最好的就是猜测最后的结果
- 大多数评估函数的工作方式是计算状态的不同特征，那么对状态的一个合理评价是加权平均值

国际象棋中EVAL通常取为加权线性函数（假设每个特征的贡献独立于其它特征的值）

$$\sum_{i=1}^n w_i f_i(s)$$

第6章 逻辑智能体

- 基于知识的智能体
 - 总体的智能体设计
- 妖兽(wumpus)世界: 一个基于知识的智能体
- 逻辑
- 命题逻辑
- 命题逻辑的推理模式
- 基于命题逻辑的智能体 (确定的知识)

基于知识的智能体

- **逻辑智能体**是**基于知识的智能体**，它采用推理过程来得到**关于新世界的表示**，并且用这些新表示推到下一步做什么。
- **基于知识的智能体**，从通用的形式表达的知识中获益，通过对信息的组合和再组合，以适应各种用途。
 - **在部分可观察的环境**，能够将常识和当前的感知结合起来，在选择行动之前推导出当前状态的隐藏部分
 - **自然语言理解**：要对隐含状态即说话者的意图进行推理
 - 具有很好的灵活性：
 - 接受新的任务：能够接受以明确描述模板的形式表示的新任务
 - 适应环境的变化：通过被告知或主动学习环境的新知识，从而快速获得能力，并可以通过更新相关知识来适应环境的变化

逻辑是用来表达信息的形式语言

- 语法：定义语言中的语句，怎样的表达是合法的

- 语义：定义语句的“意思”

在逻辑中，语义定义了每条语句关于每种可能世界的真值，非真即假

蕴涵表示一件事为真可得出另一件事也为真：

$$KB \models \alpha$$

知识库 KB 蕴涵语句 α ，当且仅当在 KB 为真的所有模型中 α 也为真

蕴涵是语句之间的关系，它是基于语义的

推理和蕴涵/蕴含

- **逻辑推理**：用蕴含推导出结论
 - **模型检验**：通过**枚举**所有可能的模型来检验KB为真的情况下 α 都为真，即 $M(KB) \subseteq M(\alpha)$
 - **可靠的**：只导出蕴含句的推理算法被称为可靠的
 - **完备性**：如果推理算法可以生成任一蕴含句，则完备。（在有限的情况下确实完备，但是常常不完备）
-
- **比喻**：KB的所有推论集合视为一个大干草堆，而语句 α 则视为一根针
 - 如果推理算法 i 可以从KB导出 α ，我们表示为： $KB \vdash_i \alpha$ ，读为 “ i 从KB导出 α ”
 - 只要 $KB \vdash_i \alpha$ ， $KB \models \alpha$ 就为真，则 i 是**合理的**
 - 只导出蕴含句的推理算法称为**可靠的**或**真值保持的**推理
 - 只要 $KB \models \alpha$ ， $KB \vdash_i \alpha$ 就为真，则 i 是**完备的**
 - 推理算法是**完备的**，如果它可以生成任一蕴含句

- 在知识库语句上直接应用推理规则以构建目标语句的证明，而无须关注模型
- 适用于模型数目庞大而证明很短

与蕴涵相关的附加概念：逻辑等价

如果两个语句在同样的模型集合中为真，则二者逻辑等价。

定义：任意两个语句 α 和 β 是等价的当且仅当它们互相蕴涵时。

$$\alpha \equiv \beta \text{ iff } \alpha \models \beta \text{ and } \beta \models \alpha$$

标准的逻辑等价	$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$	可交换性
	$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$	
	$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$	结合律
	$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$	
	$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination	双重否定消去
	$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition	逆否命题
	$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	implication elimination	蕴涵消去
	$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination	双向蕴涵消去
	$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	de Morgan	摩根律
	$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	de Morgan	
	$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$	$\wedge$ 对 $\vee$ 的分配率
	$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$	$\vee$ 对 $\wedge$ 的分配率

与蕴涵相关的附加概念：有效性（合法性）和可满足性

- **有效性**：定义：一个语句是有效的，如果在所有的模型中它都为真。
 - 有效语句也叫重言式——必定为真：每个有效语句都逻辑等价于True。
 - 演绎定理：对于任意语句 α 和 β ， $\alpha \vdash \beta$ 当且仅当 语句 $(\alpha \Rightarrow \beta)$ 是有效的。
 - 每个有效的（合法的）蕴涵语句都描述了一个有效的（合法的）推理。
- **可满足性**：定义：一个语句是可满足的，如果对于部分模型它是真值。
 - 一个句子是不可满足的如果它在任何模型中都不为真。
- **有效性与可满足性的关联**
 - α 是有效的 当且仅当 $\neg\alpha$ 不可满足【 $\neg\alpha$ 不可满足，即在任何模型中 $\neg\alpha$ 都不为真，即在任何模型中 α 都为真，即 α 是有效的】
 - α 是可满足的当且仅当 $\neg\alpha$ 不是有效的【 $\neg\alpha$ 不是有效的，即不是在所有模型中 $\neg\alpha$ 都是真的，即在某些模型中 $\neg\alpha$ 是假的，即在某些模型中 α 是真的，即 α 是可满足的】
 - $\alpha \vdash \beta$ 当且仅当语句 $(\alpha \wedge \neg\beta)$ 是不可满足的。
 - 通过归谬来证明（反证法、矛盾法）：假定语句 β 为假，将推导出和已知公理 α 的矛盾。该矛盾正好说明语句 $(\alpha \wedge \neg\beta)$ 是不可满足的。

推导和证明

应用**推理规则**得到一个证明：一系列结论直到目标语句。

1) **假言推理规则** (Modus Ponens, 拉丁文)

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

只要给定任何形式为 $\alpha \Rightarrow \beta$ 和 α 的语句，就可以推导出语句 β

例： 已知 $(\text{WumpusAhead} \wedge \text{WumpusAlive}) \Rightarrow \text{Shoot}$
和 $(\text{WumpusAhead} \wedge \text{WumpusAlive})$
就可以推导出 Shoot

2) **消去合取词**

$$\frac{\alpha \wedge \beta}{\alpha}$$

可以从合取式推导出任何合取子式。

例： $(\text{WumpusAhead} \wedge \text{WumpusAlive})$ 可以推导出 WumpusAlive

单元归结 (unit resolution) 推理规则

$$\frac{l_1 \vee \dots \vee l_k, \quad m}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k}$$

l_i 和 m 是互补文字

互补文字：
一个文字是另一个文字的否定式。
。

全归结 (full resolution) 推理规则

$$\frac{l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

l_i 和 m_j 是互补文字

合取范式 (CNF)

合取范式 (CNF): 以子句的合取式表达的语句

命题逻辑的每个语句逻辑上等价于文字析取式的合取式 (conjunction of disjunctions of literals)

例子: $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

CNF语句可作为归结过程的输入

怎样将语句转换为CNF?

- 1、消除等价词 $\Leftrightarrow$
- 2、消去蕴涵词 $\Rightarrow$
- 3、否定词 $\neg$ 内移
- 4、消去嵌套 (在可能的位置上将 $\vee$ 对 $\wedge$ 进行分配)

- 7.1 表示方法的回顾**
- 7.2 一阶逻辑的语法和语义**
- 7.3 使用一阶逻辑**
- 7.4 一阶逻辑中的知识工程**

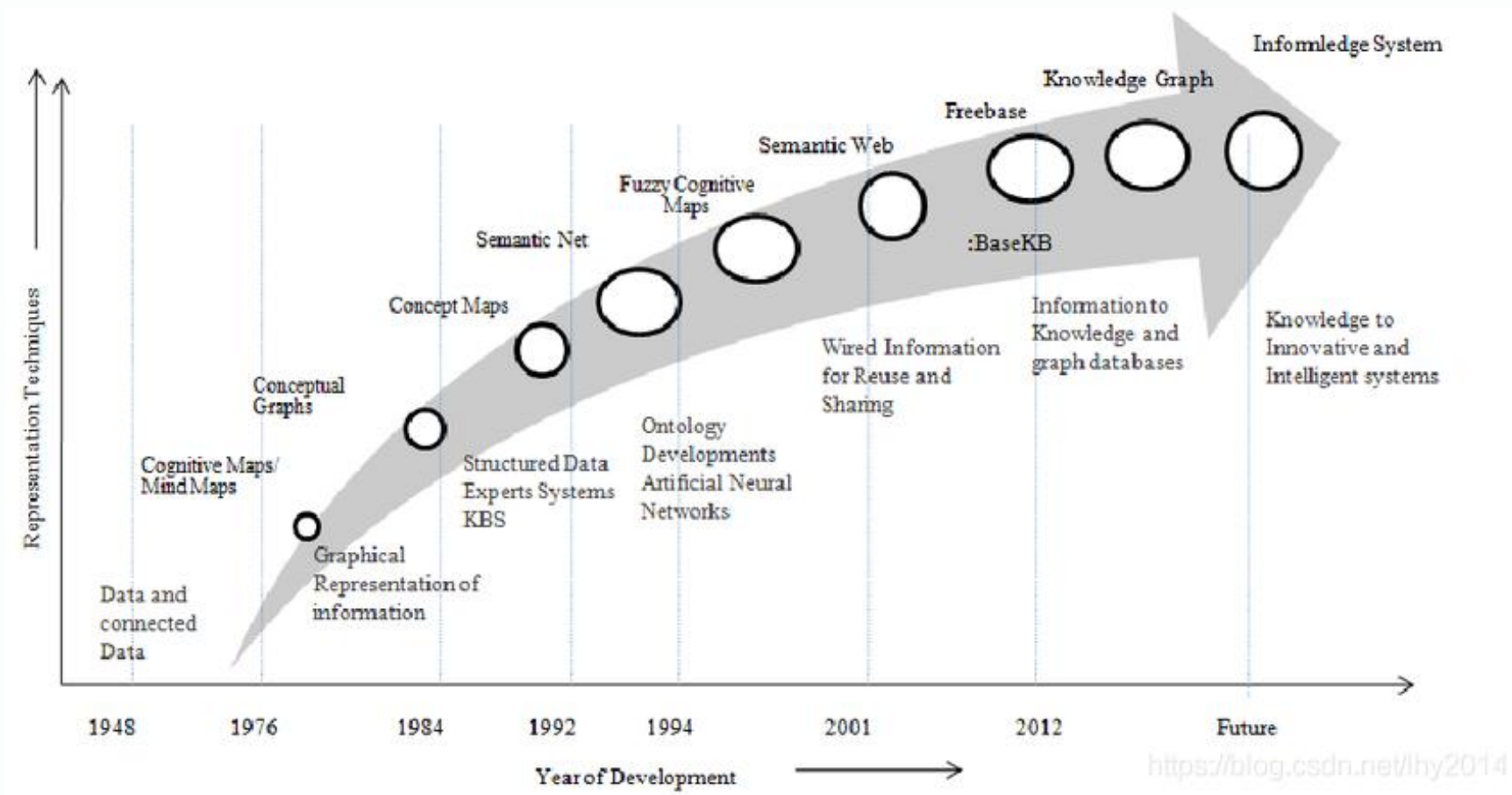
- **人工智能系统所关心的知识：**一个智能程序高水平的运行需要有关的事实知识、规则知识、控制知识和元知识。
 - **事实：**是有关问题环境的一些事物的知识，常以“...是...”的形式出现。如事物的分类、属性、事物间关系、科学事实、客观事实等，在知识库中属于低层的知识。如雪是白色的、鸟有翅膀、张三李四是好朋友。
 - **规则：**是有关问题中与事物的行动、动作相联系的因果关系知识，是动态的，常以“如果...那么...”形式出现。特别是启发式规则是属于专家提供的专门经验知识，这种知识虽无严格解释但很有用处。
 - **控制：**是有关问题的求解步骤，技巧性知识，告诉怎么做一件事。也包括当有多个动作同时被激活时应选哪一个动作来执行的知识。
 - **元知识：**是有关知识的知识，是知识库中的高层知识。包括怎样使用规则、解释规则、校验规则、解释程序结构等知识。

知识特性

- **相对正确性。**一定条件下/某种环境中
- **不确定性。**中间状态/为真程度/随机性/模糊性/经验性/不完全性
- **可表示性。**语言/文字/图像/视频/图形/音频/神经网络/概率图

知识的分类

- **范围。**常识性知识/领域性知识
- **作用。**事实性（陈述性）知识/过程性知识/控制知识
- **确定。**确定性知识/不确定性知识
- **表现。**逻辑性知识/形象性知识
- **抽象。**零级知识/一级知识/二级知识



细节上来说，知识表示主要经历了从一阶谓词逻辑表示、产生式规则、框架表示法、脚本表示法、语义网表示法、知识图谱表示法等几种表示。



- 采用命题逻辑的基础—陈述式、上下文无关、无歧义和合成语义，并借用自然语言的思想。
- 一阶逻辑假设世界包含
 - **对象**：人们、房子、数字...
 - **关系**：红色的、比...大...
 - **函数**：是...的父亲...
- 命题逻辑假设世界中的事实要么成立，要么不成立
- 一阶逻辑则假设更多，即世界由对象组成，对象之间的某些关系要么成立，要么不成立。

语句的真假由**模型**和对语句的**解释**决定

模型包含大于等于1个对象（**域成员**）和它们之间的
关系

常量符号	→	对象
谓词符号	→	关系
函数符号	→	函数关系

一条原子语句：谓词($term_1, \dots, term_n$)为真，当且仅当在给定的模型、给定的解释下 $term_1, \dots, term_n$ 代表的**对象**满足谓词所表示的**关系**

复杂语句是由原子语句通过**连接词**连接构成

$$\neg S, S_1 \wedge S_2, S_1 \vee S_2, S_1 \Rightarrow S_2, S_1 \Leftrightarrow S_2$$

全称量词 $\forall$

- 对**每个对象**进行陈述
- 形式: $\forall$ *variable sentence*, 例如
$$\forall x, King(x) \Rightarrow Person(x)$$
- $\Rightarrow$ 是使用 $\forall$ 时的**自然连接符**
- 经常犯的错误是用合取式代替蕴涵, 导致过强的陈述.
 - 例如, $\forall x, At(x, ZJU) \wedge Smart(x)$ 的含义是 “每个人都在ZJU, 而且每个人都聪明”。
 - 显然这没有抓住我们要表述的内涵。

存在量词 $\exists$

- 对全域中**某些对象**进行陈述而无需对它命名
- 例如: $\exists x, Crown(x) \wedge OnHead(x, John)$
- $\wedge$ 是使用 $\exists$ 时的自然连接符
- 在 $\exists$ 句中用 $\Rightarrow$ 代替 $\wedge$ 会导致过弱的陈述

可满足性

- 如果在所有的模型中都为真，**语句是永真的**，
 - True
 - $A \vee \neg A$
 - $A \Rightarrow A$
 - $(A \wedge (A \Rightarrow B)) \Rightarrow B$
- 语句是**可满足的**，如果在某些模型中为真，
 - $A \vee B$
 - C
- 语句是**不可满足的(永假)**，如果在所有模型中都不为真，
 - $A \wedge \neg A$

7.5 一阶逻辑的推理

- 简化为命题推理 Reduction
- 直接在一阶逻辑中推理

首先需要将量化语句用非量化语句替换（实例化语句）

量词的推理规则



○ 全称量词实例化



○ 存在量词实例化

全称量词实例化推理规则（UI规则）

对于任意变量 v 和基项（没有变量的项） g 有：

$$\frac{\forall v \quad \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

例如： $\forall x \quad \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

经过**代换**后可得

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

...

全称量词可多次实例化得到多个不同的结果

存在量词实例化推理规则 (EI规则)

对于任意语句的 α , 变量 v , 以及从未在知识库中出现过的常量符号 k 有:

$$\frac{\exists v \quad \alpha}{\text{SUBST}(\{v/k\}, \alpha)}$$

例如:

$$\exists x \quad \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

经过置换 $\{x/C_1\}$ 后可得

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

一般化的假言推理规则

广义假言推理

- 一般化的假言推理规则将假言推理规则从命题逻辑提升到一阶逻辑

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

- 代换 θ 使得
 $\text{SUBST}(\theta, p_1' \wedge \dots \wedge p_n') = \text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n)$

找到使不同的逻辑表示变得相同的代换

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 $\text{King}(\text{John})$
 $\text{Greedy}(\text{John})$
 $\text{Brother}(\text{Richard}, \text{John})$

https://blog.csdn.net/weixin_44972129

合一 Unification

合一：

- GMP推理规则中找到相关置换的过程。
- $\text{Unify}(\alpha, \beta) = \theta$ if $\text{subst}(\alpha, \theta) = \text{subst}(\beta, \theta)$

□ 找到使不同的逻辑表示变得相同的代换的过程称为**合一**。**合一算法**Unify以两条语句为输入，如果存在合一代换则返回该**合一代换（合一元）**，否则返回失败。

$$\text{UNIFY}(p, q) = \theta, \text{ where } \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x / \text{Jane}\}$$

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Bill})) = \{x / \text{Bill}, y / \text{John}\}$$

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Mother}(y))) = \{y / \text{John}, x / \text{Mother}(\text{John})\}$$

https://blog.csdn.net/weixin_44972129

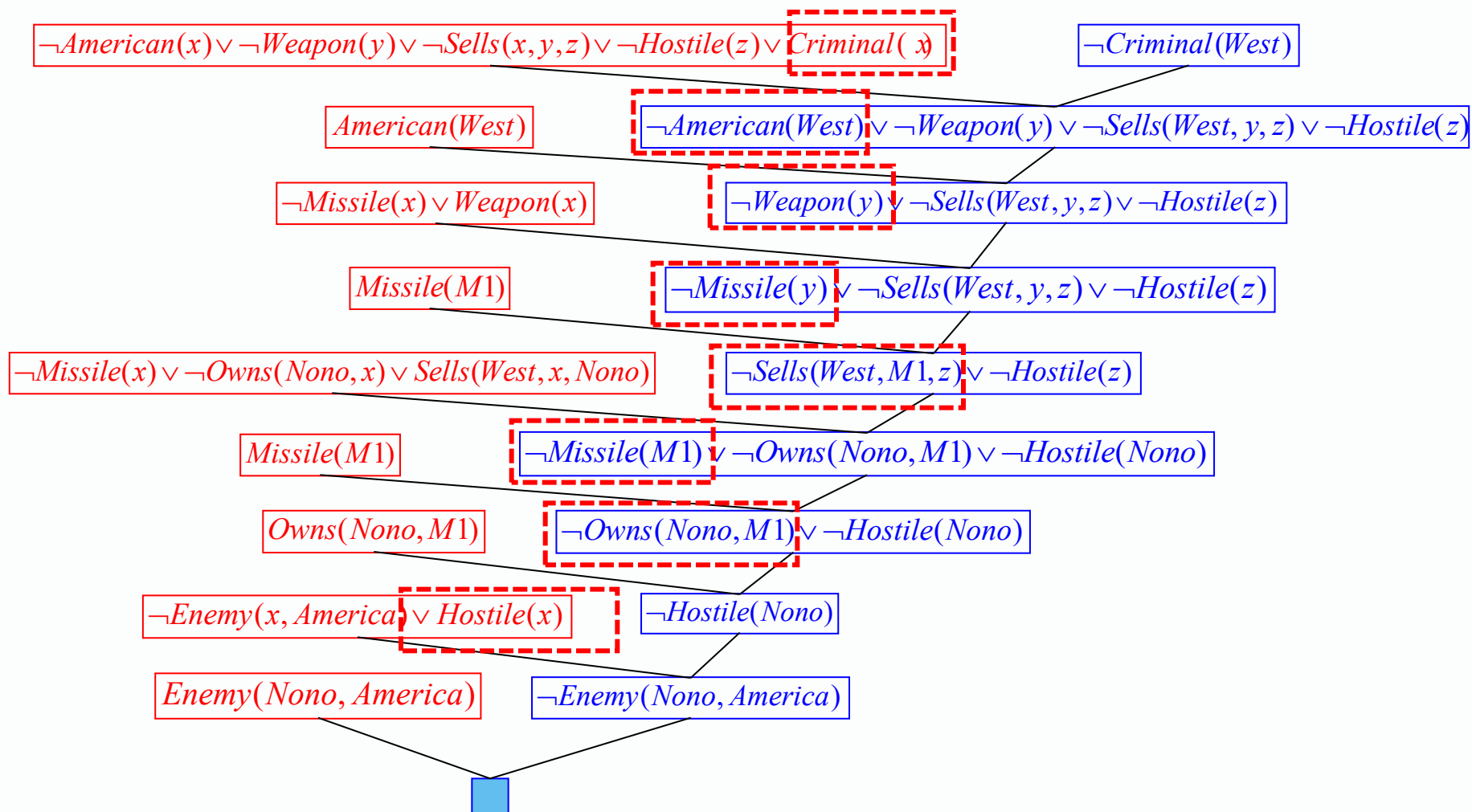
最一般合一代换MGU

□ 合一代换可能不唯一

- UNIFY (Knows(John,x), Knows(y,z))
- $\{y/\text{John}, x/z\}$ or $\{x/\text{John}, y/\text{John}, z/\text{John}\}$
- Knows (John, z) or Knows (John, John)
- First unifier is more **general** than second because it places fewer restrictions on the values of the variables

- There is a *most general unifier (MGU)* for every unifiable pair of expressions

归结推理证明



机器学习



需掌握的核心内容

• 机器学习概述

- 机器学习概念定义
- 学习问题定义
- 学习问题的类别

• 概念学习

- 概念学习定义
- 概念学习任务
- 归纳学习与归纳学习假设
- Find-S: 寻找极大特殊假设
- 变形空间和候选消除算法
- 归纳偏置

• 决策树

- 决策树定义
- 适用问题特征

- 基本ID3算法
- 决策树学习的归纳偏置
- 训练数据的过度拟合
- 奥坎姆剃刀原理
- C4.5算法

• 神经网络

- 神经网络定义与概念
- 感知器 (perceptron)
- 线性单元 (linear unit)
- 训练多层网络的反向传播算法 (Delta法则)
- ANN的表征能力
- 假设空间搜索的本质特征
- 神经网络过度拟合问题
- CNN
- RNN, LSTM

• 贝叶斯学习

- 贝叶斯理论概念定义
- 极大似然假设 (ML) 和极大后验概率假设 (MAP)
- 贝叶斯理论对其他学习算法的解释
- 贝叶斯最优分类器
- Gibbs算法
- 朴素贝叶斯分类器

• 强化学习

- 与其他学习方法的异同点
- 马尔科夫决策过程MDP
- Q-Learning, Sarsa, Policy Gradients
- Actor-Critic,
- DQN

• 算法评估

- 假设的评估、两个假设精度的比较、两个学习算法精度比较
- 样本错误率、真实错误率
- 交叉验证与t配对测试



概述

- 机器学习的概念和定义
 - 表达 + 性能评估 + 优化
 - 使得计算机具备和人类一样的学习能力
 - 给定数据（样本、实例）和一定的学习规则，从数据中获取知识的能力
- 学习问题
 - 学习问题的类别
 - 根据训练数据是否有标记
 - 监督学习（有监督学习） -- 分类问题
 - 非监督学习（无监督学习） -- 聚类问题
 - 学习问题的定义
 - 对某类任务 T 和性能度量 P ，如果一个计算机程序在 T 上以 P 衡量的性能随着经验 E 而完善，那么我们称这个计算机程序在从经验 E 中学习
 - 定义学习问题的三个特征
 - 任务 T
 - 性能度量 P
 - 经验 E

概述

机器学习的概念和定义

- 表达+性能评估+优化
- 使得计算机具备和人类一样的学习能力
- 给定数据（样本、实例）和一定的学习规则，从数据中获取知识的能力

学习问题

- 学习问题的类别 —— 根据训练数据是否有标记
 - 监督学习（有监督学习） —— 分类问题
 - 非监督学习（无监督学习） —— 聚类问题
- 学习问题的定义 —— 对某类任务 T 和性能度量 P ，如果一个计算机程序在 T 上以 P 衡量的性能随着经验 E 而完善，那么我们称这个计算机程序在从经验 E 中学习
- 定义学习问题的三个特征
 - 任务 T
 - 性能度量 P
 - 经验 E

概念学习

- **概念学习定义**

- 机器学习的中心任务
 - 从特殊的训练样例中归纳出一般函数
- 从有关某个布尔函数的输入输出的训练集中，推测出这个布尔函数。
- 有关术语
 - 概念、目标概念
 - 实例、实例空间
 - 训练样例
 - 正例、反例
 - 假设、假设空间
 - 属性、属性值

概念学习定义



机器学习的中心任务

从特殊的训练样例中归纳出一般函数



从有关某个布尔函数的输入输出的训练集中，推测出这个布尔函数。

有关术语

概念、目标概念

实例、实例空间

训练样例

正例、反例

假设、假设空间

属性、属性值

概念学习

- 概念学习任务
 - 一般来说，任何概念学习任务能被描述为
 - 实例的集合
 - 实例集合上的目标任务（目标概念）
 - 训练样例的集合
 - 候选假设的集合
 - 形式化描述
 - 已知
 - 实例集 X :每个实例 x 由若干个属性描述，每个属性的取值范围已确定
 - 假设集 H :每个假设 h 描述为上述各个属性的取值约束的合取
 - 目标概念 c :一个布尔函数，变量为实例
 - 训练样例集 D :目标函数（或目标概念）的正例和反例
 - 求解
 - H 中的一假设 h ，使对于 X 中任意 x ， $h(x) = c(x)$
 - 两种概念学习任务
 - 分类
 - 目标标记为类别型数据（离散型）
 - 回归
 - 目标标记为连续型数值

概念学习任务

✓ 一般来说，任何概念学习任务能被描述为

- 实例的集合
- 实例集合上的目标任务（目标概念）
- 训练样例的集合
- 候选假设的集合

💡 形式化描述

已知

- 实例集 X : 每个实例 x 由若干个属性描述，每个属性的取值范围已确定
- 假设集 H : 每个假设 h 描述为上述各个属性的取值约束的合取
- 目标概念 c : 一个布尔函数，变量为实例
- 训练样例集 D : 目标函数（或目标概念）的正例和反例

求解

H 中的一假设 h ，使对于 X 中任意 x ， $h(x)=c(x)$

两种概念学习任务

分类

—— 目标标记为类别型数据（离散型）

回归

—— 目标标记为连续型数值

概念学习

- 归纳学习与归纳学习假设

- 基础概念
 - 归纳学习
 - 广义 从样例中学习：从特殊到一般的“泛化”过程 (generalization)
 - 狭义 要求从训练数据中学得概念 (concept)，也称概念学习，或概念形成
 - 归纳学习假设
 - 如果没有更多的信息，对于未见实例最好的假设就是与训练数据最佳拟合的假设
- 概念学习可以看作一个搜索的过程
 - 搜索范围：假设的表示所隐含定义的整个空间
 - 搜索目标：能够最好地拟合训练样例的假设
- 假设的“一般到特殊序”
 - 关系“.....比.....更一般”
 - 任给实例 x 和假设 h ，称 x 满足 h ，当且仅当 $h(x)=1$
 - “更一般”的严格情形
 - “更特殊” (more specific than) 关系的定义
- 对假设空间进行搜索的策略
 - 搜索过程中不断删除①与正例不一致的假设②与反例一致的假设，最终得到与训练集一致（即对所有训练样本能够进行正确判断）的假设
 - 自底向上，从特殊到一般
 - 自顶向下，从一般到特殊

✓ 基础概念

归纳学习

- 广义 —— 从样例中学习：从特殊到一般的“泛化”过程 (generalization)
- 狭义 —— 要求从训练数据中学得概念 (concept)，也称概念学习，或概念形成



归纳学习假设

如果没有更多的信息，对于未见实例最好的假设就是与训练数据最佳拟合的假设

概念学习可以看作一个搜索的过程

搜索范围：假设的表示所隐含定义的整个空间

搜索目标：能够最好地拟合训练样例的假设

假设的“一般到特殊序”

关系“.....比.....更一般” —— 任给实例 x 和假设 h ，称 x 满足 h ，当且仅当 $h(x)=1$

“更一般”的严格情形

“更特殊” (more specific than) 关系的定义

对假设空间进行搜索的策略

搜索过程中不断删除①与正例不一致的假设②与反例一致的假设，最终得到与训练集一致（即对所有训练样本能够进行正确判断）的假设

自底向上，从特殊到一般

自顶向下，从一般到特殊

概念学习

- Find-S:寻找极大特殊假设
 - 寻找极大特殊假设。从H中最特殊假设开始，然后在假设覆盖正例失败时将其一般化。
 - 特点：对以属性约束的合取式描述的假设空间H，保证输出为H中与正例一致的最特殊的假设。
- 变型空间（版本空间）与候选消除算法
 - 变型空间（版本空间）
 - “一致”
 - 与训练集一致的假设集合
 - 列表后消除算法
 - 从变型空间中去除与任一训练样例不一致的假设
 - 优点：保证得到所有与训练数据一致的假设
 - 缺点：非常繁琐地列出H中的所有假设，大多数实际的假设空间无法做到
 - 候选消除算法
 - 使用更简洁的变型空间
 - 一般边界G，是在H中与D相一致的极大一般成员的集合
 - 特殊边界S，是在H中与D相一致的极大特殊成员的集合
 - G和S之间偏序结构所规定的假设
 - 输出与训练样例一致的所有假设的集合
 - 优点：利用more_general_than偏序结构，不需要明确列举所有成员
 - 缺点：缺少健壮性（Robustness，鲁棒性）

概念学习

- 归纳偏置

- 关于目标函数的必要假设
- 3个有偏程度不同的归纳学习算法
 - 机械式--没有归纳偏置
 - 候选消除算法--有较强的归纳偏置（目标概念可以在假设空间中找到）
 - Find-S--有更强的归纳偏置

决策树

- 决策树定义
 - 一种非参数的有监督学习方法，它能够从一系列有特征和标签的数据中总结出决策规则，并用树状图的结构来呈现这些规则，以解决分类和回归问题。
 - 决策树的组成部分
 - 决策结点
 - 分支
 - 叶子
 - 核心任务
 - 把样例分类到各可能的离散值对应的类别
 - 决策树学习过程
 - 自根至叶的递归过程，在每个中间结点寻找一个“划分” (splitortest)属性。
 - 特征选择（选择决策节点）
 - 决策树生成
 - 剪枝
- 适用问题特征
 - 实例是由“属性-值”对 (pair) 表示的
 - 目标函数具有离散的输出值
 - 可能需要析取的描述
 - 训练数据可以包含错误训练数据
 - 可以包含缺少属性值的实例

决策树

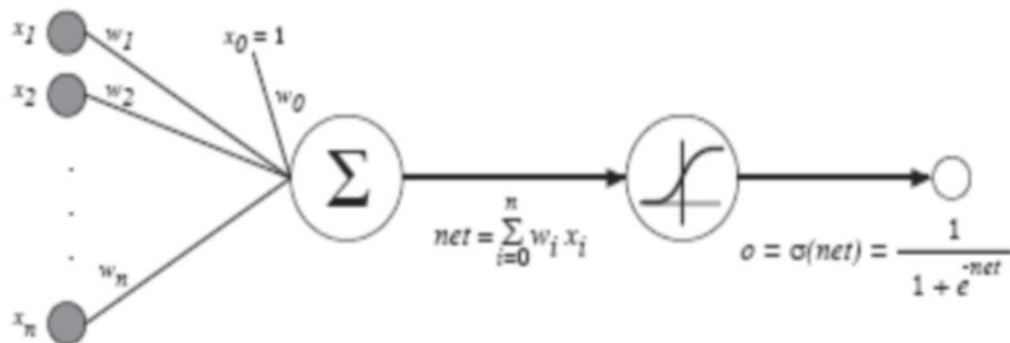
- 基本ID3算法
 - 通过自顶向下构造决策树来进行学习，构造过程从“哪一个属性将在树的根节点被测试”这个问题开始。
 - 基本思想：以信息熵为度量，用于决策树节点的属性选择，每次优先选取信息量最多的属性，亦即使熵值变为最小的属性，以构造一颗熵值下降最快的决策树，到叶子节点处的熵值为0。此时，每个叶子节点对应的实例集中的实例属于同一类。
 - 核心问题：选取在树的每个节点要测试的属性，即属性选择问题
 - 信息增益：在划分数据集之前之后信息发生的变化
 - 信息论
 - » 信息熵、条件熵、经验熵、经验条件熵
 - 计算步骤
 - » 1、计算数据集D的经验熵 $H(D)$
 - » 2、计算特征A对数据集D的经验条件熵 $H(D|A)$
 - » 3、计算互信息：熵 $H(D)$ 与条件熵 $H(D|A)$ 之差
 - 信息增益代表了在一个条件下，信息复杂度(不确定性)减少的程度。
 - ID3算法步骤
 - 决定分类属性
 - 1、创建决策树的根节点（选择分类能力最好的属性）
 - 选择该特征后，信息增益最大(信息不确定性减少的程度最大)
 - 2、在根节点下创建分支，得到部分决策树
 - 3、将训练样例排列到适当的分支
 - 4、重复上述过程

决策树

- 决策树学习的归纳偏置
 - ID3算法搜索的假设空间包含所有的决策树。
 - ID3以爬山算法遍历这个假设空间，引导这种爬山搜索的评估函数是信息增益度量
 - 较短的树比较长的树优先，信息增益高的属性更靠近根节点的树优先
- 奥坎姆剃刀原理
 - 优先选择拟合数据的最简单的假设
- 训练数据的过度拟合
 - 原因
 - 当数据中有噪声或训练样例的数量太少
 - 防止/缓解
 - 剪枝
 - 前剪枝
 - 后剪枝
- C4.5算法
 - 信息增益比：信息增益除以属性A的不确定度
 - 对可取数值数目较少的属性有所偏好

神经网络

- 神经网络定义与概念
 - 神经网络是一个具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应。
 - 适合神经网络学习的问题
 - 训练集合为含有噪声的复杂传感器数据
 - 需要较多符号表示的问题
- 感知器 (perceptron)
 - 一种具有单层计算单元的神经网络
 - 可以把感知器看作是n维实例空间（即点空间）中的超平面决策面
 - 单层感知器：仅对线性问题具有分类能力，无法处理非线性问题
 - 多层感知器：在输入层和输出层之间加入隐层，以形成能够将样本正确分类的凸域
- 线性单元 (linear unit) 和Sigmoid单元



神经网络

- 权值更新法则
 - 感知器法则：根据阈值化的感知器输出的误差更新权值

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta(t - o)x_i$$

- Delta法则：根据输入的非阈值化线性组合的误差来更新权值

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

$$\vec{w} \leftarrow \vec{w} + \Delta \vec{w}$$

$$\Delta \vec{w} = -\eta \nabla E(\vec{w})$$

- 训练多层网络的反向传播算法 (Delta法则)
 - 反向传播算法
 - 采用梯度下降方法，**试图最小化网络输出值和目标值之间的误差平方**
 1. 从建立一个具有期望数量的隐层单元和输出单元的网络并初始化所有的网络的权值为小的随机数开始
 2. 给定一个固定的网络结构，算法的主循环就对训练样例进行反复的迭代
 3. 对于每一个训练样例，它应用目前的网络到这个样例，计算出对这个样例网络输出的误差，然后更新网络中所有的权值
 4. 对这样的梯度下降步骤进行迭代，直到网络的性能达到可接受的精度为止

• 梯度下降法则

与delta训练法则相似，误差项更为复杂

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{output}} (t_{kd} - o_{kd})^2$$

$$\frac{\partial E_d}{\partial \text{net}_j}$$

随机梯度下降法则的推导

输出单元

$$\frac{\partial E_d}{\partial \text{net}_j} = -(t_j - o_j) o_j (1 - o_j)$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j (1 - o_j) x_{ji}$$

隐层单元

$$\frac{\partial E_d}{\partial \text{net}_j} = -o_j (1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj}$$

$$\Delta w_{ji} = -\eta x_{ji} o_j (1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj}$$

• 特性

- 梯度下降仅能保证收敛到局部极小值
- 对于实践中很多应用，反向传播算法都产生了出色的结果
- 太少的迭代无法有效地降低误差，太多的迭代会导致对训练数据的过度拟合
- 能够在网络内部的隐藏层发现有用的中间表示
- 网络中使用的单元层越多，就可以创造出越复杂的特征

- ANN的表征能力

- 布尔函数：任何布尔函数可以被具有两层单元的网络准确表示
- 连续函数：每个有界的连续函数可以由一个两层的网络以任意小的误差逼近
- 任意函数：任意函数可以被一个有三层单元的网络以任意精度逼近

- 假设空间搜索的本质特征

- 反向传播算法的假设空间是 n 个网络权值形成的 n 维欧氏空间。这个空间是连续的
- 精确地刻画出反向传播学习的归纳偏置有难度
- 把这一偏置粗略地刻画为在数据点之间平滑插值。如果给定两个正例，它们之间没有反例，反向传播算法会倾向于把这两点之间的点也标记为正例

神经网络

- 用来缓解局部极小值问题的启发式规则
 - 为梯度更新法则加一个冲量
 - » 梯度下降的搜索轨迹就像一个球沿误差曲面滚下，冲量使球从一次迭代到下一次迭代时以同样的方向滚动
 - » 冲量有时会使这个球滚过误差曲面的局部极小值或平坦区域（原则上，也可能冲过狭窄的全局最小值）
 - » 冲量也具有在梯度不变的区域逐渐增大搜索步长的效果，从而加快收敛
 - 使用随机的梯度下降
 - 使用同样的数据训练多个网络，但用不同的随机权值初始化每个网络

神经网络

- 神经网络过度拟合问题
 - 拟合了训练数据中的噪声和训练样例中没有代表性的特征
 - 解决办法
 - 权值衰减
 - 使学习过程向着复杂决策面的反方向偏置
 - 验证数据
 - 交叉验证
 - » k-fold交叉验证
 - 较少的隐层单元可防止网络过度拟合数据
 - 应用交叉验证方法确定隐层单元数
 - 交叉验证：可以用来估计梯度下降搜索的合适终止点，从而最小化过度拟合的风险
- 学习率 η
 - 神经网络中最重要的超参数
 - 更大的学习率会收敛得更快，但是容易出现震荡而无法收敛
 - 更小的学习率需要模型更多次的迭代，耗时

神经网络（深度学习）



深度学习的起源与里程碑

深度学习起源于人工神经网络的研究，1958年感知机模型的提出标志着深度学习的起点。2012年，AlexNet在ImageNet竞赛中取得突破性成绩，开启了深度学习在图像识别领域的广泛应用。



神经网络的基本概念

神经网络由输入层、隐藏层和输出层组成，通过调整权重和偏置实现非线性拟合。激活函数如ReLU、Sigmoid等在网络中扮演着至关重要的角色，增强了网络的表达能力。



深度学习与传统机器学习

深度学习通过构建多层网络结构，能够自动提取特征，减少人工干预。与传统机器学习相比，深度学习在图像、语音识别等领域展现出更强大的性能。

卷积神经网络（CNN）原理

卷积层的作用与机制

卷积层通过滤波器捕捉输入数据的空间特征，实现特征的局部感知和权重共享。

卷积操作包括卷积核滑动、元素乘积和求和，有效提取图像中的边缘、纹理等特征。

池化层的作用与类型

池化层通过下采样减少数据维度，降低计算量，增强特征的平移不变性。

池化类型包括最大池化和平均池化，分别提取局部最大值和平均值，减少过拟合。

全连接层与输出层

全连接层将卷积层和池化层提取的特征映射到最终的输出空间，实现分类或回归任务。

输出层通常使用Softmax函数进行多分类，输出每个类别的概率分布。

循环神经网络（RNN）基础

RNN的循环结构

01

RNN通过循环结构处理序列数据，每个时间步的输出依赖于当前输入和前一时间步的输出。
RNN的核心在于隐藏状态的传递，能够捕捉序列中的长期依赖关系。

RNN的变体与优化

02

由于梯度消失和梯度爆炸问题，RNN在处理长序列时性能受限。
LSTM和GRU等变体通过引入门控机制解决了这些问题，提高了RNN的稳定性和性能。

RNN的应用场景

03

RNN在自然语言处理、语音识别等领域有广泛应用，如语言模型、机器翻译等。
RNN能够处理时间序列数据，捕捉时间上的动态变化，适用于预测和分类任务。

贝叶斯学习

- 贝叶斯理论对其他学习算法的解释

- 对其他不直接应用贝叶斯公式的学习方法的分析提供理论基础

- 给定训练数据，最可能的假设是什么？

- 贝叶斯法则为计算给定训练数据下任一假设的后验概率提供了原则性方法，我们可直接把它作为一个基本的学习算法，计算每个假设的后验概率，再输出其中后验概率最大的。

- 极大似然与最小误差平方和假设

- 在**特定前提**下，任一学习算法如果使输出的假设预测和训练数据之间的误差平方最小化，它将输出一极大似然假设。

- 极大似然与最小交叉熵假设

- 最小描述长度准则MDL

- 建议选取这样的假设，它使假设的描述长度和给定假设下数据的描述长度的和最小化

$$\begin{aligned}h_{ML} &= \arg \max_{h \in H} P(D|h) \\&= \arg \max_{h \in H} \prod_{i=1}^m \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(d_i - f(x_i))^2} \\&= \arg \max_{h \in H} \prod_{i=1}^m e^{-\frac{1}{2\sigma^2}(d_i - h(x_i))^2}\end{aligned}$$



取对数

$$\begin{aligned}h_{ML} &= \arg \max_{h \in H} \sum_{i=1}^m \ln \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2\sigma^2}(d_i - h(x_i))^2 \\&= \arg \min_{h \in H} \sum_{i=1}^m (d_i - h(x_i))^2\end{aligned}$$

贝叶斯学习

Bayes optimal classification:

$$\arg \max_{v_j \in V} \sum_{h_i \in H} P(v_j | h_i) P(h_i | D)$$

- **贝叶斯最优分类器**

- 给定训练数据，对新实例的最可能的分类是什么？
- 计算开销很大：需要计算H中每个假设的后验概率，然后合并每个假设的预测来分类新实例。

- **Gibbs算法**

- 替代bayes，但非最优
 - 1、按照H上的后验概率分布，从H中随机选择假设h
 - 2、使用h来预测新实例x的分类。

Gibbs algorithm:

1. Choose one hypothesis at random, according to $P(h|D)$
2. Use this to classify new instance

- **朴素贝叶斯分类器**

- 基于一个简单的假定：在给定目标值时属性值之间相互条件独立
 - 学习
 - 分类

- **贝叶斯信念网**

- 条件独立性
- 通过贝叶斯链式法则表示变量间的联合概率分布。
 - 表征
 - 推理
 - 学习

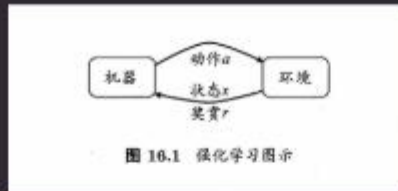
Naive Bayes assumption:

$$P(a_1, a_2 \dots a_n | v_j) = \prod_i P(a_i | v_j)$$

which gives

Naive Bayes classifier: $v_{NB} = \arg \max_{v_j \in V} P(v_j) \prod_i P(a_i | v_j)$

强化学习



强化学习任务通常用马尔可夫决策过程 (Markov Decision Process, 简称 MDP) 来描述

任务与奖赏

要找到能使长期累积奖赏最大化的策略

学习目的

单步强化学习

K-摇臂赌博机

机器已经对环境进行建模, 能在机器内部模拟出与环境相同或近似的状况

有模型学习

学习算法不依赖于环境建模

蒙特卡洛强化学习

免模型学习

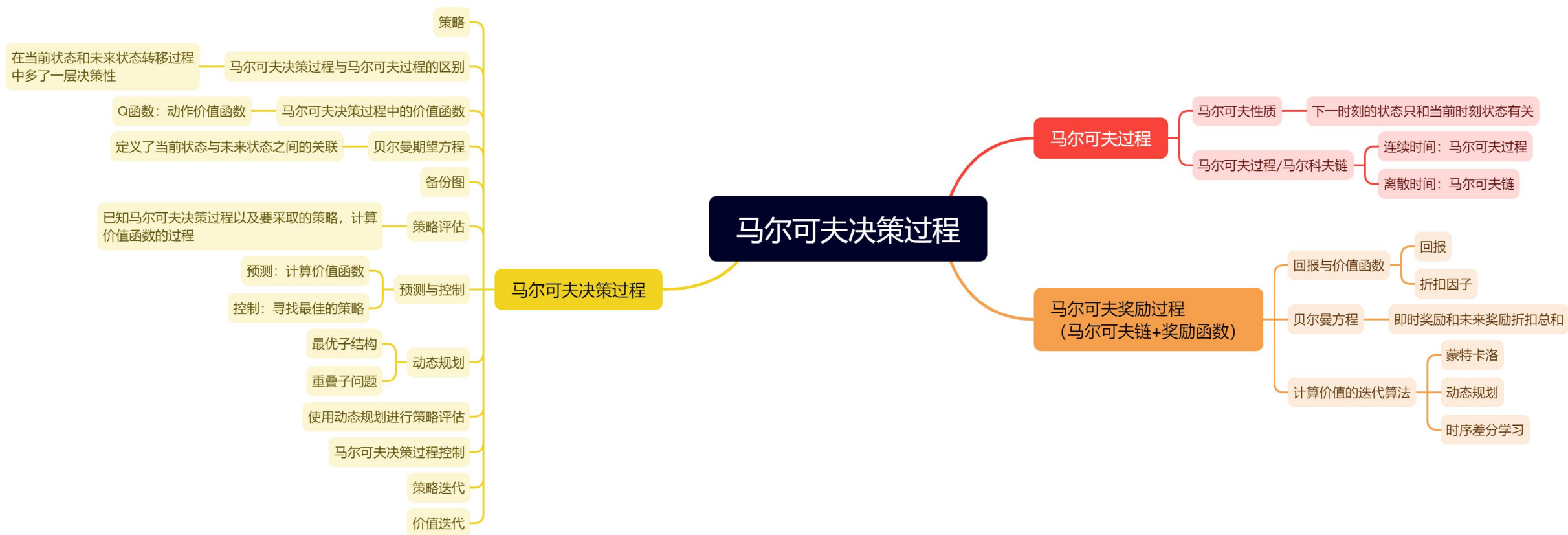
时序差分强化学习

直接模仿学习

模仿学习

逆强化学习

第十六章 强化学习



基于价值

Q-Learning

基于概率

Policy Gradients

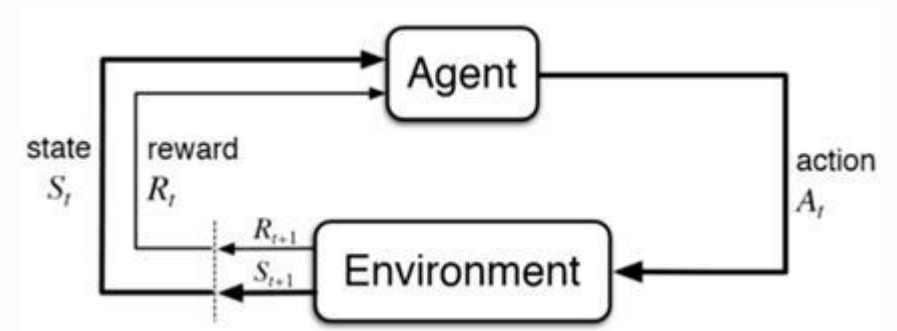
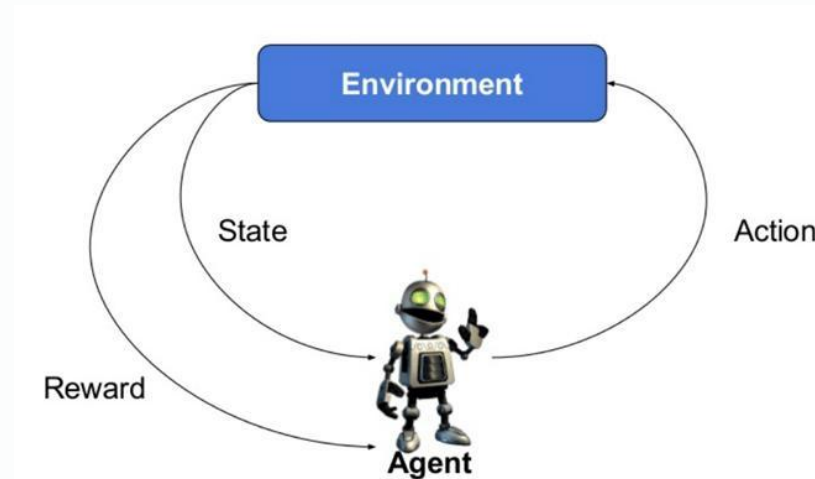
两者结合

Actor Critic

Sarsa

结合深度网络

DQN



关键概念？

贝叶斯学习

- 贝叶斯理论概念定义

- 贝叶斯决策理论

- 1、已知类条件概率密度参数表达式和先验概率。
 - 2、利用贝叶斯公式转换成后验概率。
 - 3、根据后验概率大小进行决策分类。

- 贝叶斯定理/贝叶斯法则

- 极大似然假设 (ML) 和极大后验概率假设 (MAP)

- MAP

- ML

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

Generally want the most probable hypothesis given the training data

Maximum a posteriori hypothesis h_{MAP} :

$$\begin{aligned} h_{MAP} &= \arg \max_{h \in H} P(h|D) \\ &= \arg \max_{h \in H} \frac{P(D|h)P(h)}{P(D)} \\ &= \arg \max_{h \in H} P(D|h)P(h) \end{aligned}$$

If assume $P(h_i) = P(h_j)$ then can further simplify, and choose the *Maximum likelihood* (ML) hypothesis

$$h_{ML} = \arg \max_{h_i \in H} P(D|h_i)$$

算法评估

- 如何评估一个学习算法在给定问题上的期望误差率
 - 样本错误率、真实错误率
 - 置信区间
 - **中心极限定理**
 - **任意样本均值的估计量服从的分布在n足够大时可以近似为正态分布**
- 学习算法比较：给定两个学习算法，如何就给定应用来判断一个算法的误差率比另一个低
 - 比较平均性能：从分布D中抽取的所有样本S上计算的期望值
 - 只有一个有限的样本D0，把它分割成训练集合S0和测试集合T0
 - K-Fold交叉验证：先将数据集D划分为k个大小相似的互斥子集，每次采用k-1个子集的并集作为训练集，剩下的那个子集作为测试集。进行k次训练和测试，最终返回k个测试结果的均值。
- t配对测试
 - 在同样的测试集上测试两个假设 h_A, h_B
 - 估计一系列独立同正态分布的随机变量的样本均值

Paired t test to compare h_A, h_B

1. Partition data into k disjoint test sets $T_1, T_2, \dots, T_k$ of equal size, where this size is at least 30.

2. For i from 1 to k , do

$$\delta_i \leftarrow \text{error}_{T_i}(h_A) - \text{error}_{T_i}(h_B)$$

3. Return the value $\bar{\delta}$, where

$$\bar{\delta} \equiv \frac{1}{k} \sum_{i=1}^k \delta_i$$